

Mathematical Notes for uwnav_dynamics

Project uwnav_dynamics

April 14, 2026

Contents

1	Notation	3
1.1	Discrete Time and Sampling Axis	3
1.2	Raw Sensor Streams	3
1.3	Causal Proxy State and Context	3
1.4	Model Input and Supervision	4
1.5	Prediction Outputs	4
2	Data Preprocessing and Proxy-State Construction	5
2.1	Why Preprocessing Must Be Part of the Theory	5
2.2	Raw Sensor Streams	5
2.3	IMU Preprocessing	6
2.4	Multi-Rate Alignment	6
2.5	Causal KF / ESKF Fusion	7
2.6	From Raw Observations to Dense Proxy States	7
2.7	Full-Run Visual Audit	7
2.8	Training Precursor Interpretation	7
3	Underwater Dynamics Model and Motivation	9
3.1	Classical Dynamics Background	9
3.2	Control-Oriented Identification Objective	9
3.3	Why Proxy-State Learning Is More Representative Than Raw-Sensor Fitting . .	10
3.4	Role of Causal Filtering	10
3.5	Weak Structural Priors	10
4	Neural Dynamics Model	11
4.1	History Encoder	11
4.2	Grouped Prediction Head	11
4.3	Multi-Step Block Predictor	11
4.4	Single-Step Transition Branch	11
4.5	Optional Structured Modules	12
4.6	Architectural Interpretation	12
5	Training Objective	12
5.1	Normalized Training Space	12
5.2	Gaussian Output Objective	12
5.3	Mask-Aware Supervision	13
5.4	Transition-Balance Objective	13
5.5	Single-Step Interpretation	13
5.6	Model Selection Criterion	13

6	Feasibility of a Learned Transition Solver	14
6.1	Problem Statement	14
6.2	Why Sensor Preprocessing is Necessary	14
6.3	Causal Filtering Makes the Learning Problem Better Posed	15
6.4	Why Proxy-State Training Is More Representative	15
6.5	Role of DVL, IMU, and Power	16
6.6	Why Shared Scaling is a Hard Requirement	16
6.7	Phase 1 Feasibility Claim	16
6.8	Current Engineering Interpretation	17
6.9	Next Mathematical Step	17
7	Control Integration	17
7.1	Target Role of the Learned Model	17
7.2	Why Offline Screening Is Still Necessary	18
7.3	Controller-Oriented Objective	18
7.4	Why the Single-Step Branch Matters	18
7.5	Current Boundary of Claims	18
8	Interface to Control Algorithms	19
8.1	Minimal Solver API	19
8.2	Diagnostic Information	19
8.3	Compatibility With Controller and RL Loops	19
8.4	Current Repository Status	20

1 Notation

1.1 Discrete Time and Sampling Axis

Let the aligned main timeline be indexed by $k \in \mathbb{Z}_{\geq 0}$ with sampling interval

$$\Delta t = 0.01 \text{ s}, \quad (1)$$

which corresponds to the current 100 Hz master axis.

We use:

- L : history window length,
- H : prediction horizon length,
- D_x : state dimension,
- D_c : context dimension,
- D_{in} : model input dimension,
- D_{out} : supervised target dimension.

1.2 Raw Sensor Streams

At discrete time k , let

$$\mathbf{u}_k \in \mathbb{R}^8 \quad (2)$$

denote the thruster-control command vector represented by eight PWM channels.

The raw multi-rate observations are denoted abstractly by

$$\mathcal{Y}_k = \left\{ \mathbf{y}_k^{\text{imu}}, \mathbf{y}_k^{\text{dvl}}, \mathbf{y}_k^{\text{pow}} \right\}, \quad (3)$$

where IMU is high-rate, DVL is sparse low-rate, and Power is low-rate auxiliary sensing.

1.3 Causal Proxy State and Context

The current project does not train directly on raw asynchronous observations. Instead, a causal filtering stage produces a proxy state

$$\hat{\mathbf{x}}_k = \begin{bmatrix} \hat{\mathbf{a}}_k \\ \hat{\boldsymbol{\omega}}_k \\ \hat{\mathbf{v}}_k \end{bmatrix} \in \mathbb{R}^{D_x}, \quad D_x = 9, \quad (4)$$

where:

- $\hat{\mathbf{a}}_k \in \mathbb{R}^3$ is the body-frame linear-acceleration proxy,
- $\hat{\boldsymbol{\omega}}_k \in \mathbb{R}^3$ is the body-frame angular-rate proxy,
- $\hat{\mathbf{v}}_k \in \mathbb{R}^3$ is the body-frame velocity proxy.

The auxiliary context is denoted by

$$\mathbf{c}_k = [\phi_k \quad \theta_k \quad \sin \psi_k \quad \cos \psi_k \quad \mathbf{q}_k \quad \mathbf{p}_k]^\top, \quad (5)$$

where:

- ϕ_k, θ_k, ψ_k are attitude terms,

- $\mathbf{q}_k$ denotes state-quality context,
- $\mathbf{p}_k \in \mathbb{R}^8$ denotes motor power observations.

For the current quality-context branch,

$$\mathbf{q}_k = \begin{bmatrix} h_k & \delta_k & \sigma_{v_x,k}^2 & \sigma_{v_y,k}^2 & \sigma_{v_z,k}^2 \end{bmatrix}^\top, \quad (6)$$

where h_k indicates whether a DVL update is present, δ_k denotes the elapsed time since the last DVL correction, and the last three terms denote velocity-state variance proxies.

1.4 Model Input and Supervision

The aligned model-side feature vector is

$$\mathbf{z}_k = \begin{bmatrix} \mathbf{u}_k \\ \hat{\mathbf{x}}_k \\ \mathbf{c}_k \end{bmatrix} \in \mathbb{R}^{D_{\text{in}}}. \quad (7)$$

The current active branches use:

- $D_{\text{in}} = 29$ for the baseline KF-context branch,
- $D_{\text{in}} = 34$ for the quality-context and single-step branches,
- $D_{\text{out}} = D_x = 9$ for all active transition branches.

Given a history window of length L , the model input is

$$\mathbf{X}_k^{(L)} = \begin{bmatrix} \mathbf{z}_{k-L+1} \\ \vdots \\ \mathbf{z}_k \end{bmatrix} \in \mathbb{R}^{L \times D_{\text{in}}}. \quad (8)$$

The future supervision window is

$$\mathbf{Y}_{k+1:k+H}^{(H)} = \begin{bmatrix} \hat{\mathbf{x}}_{k+1} \\ \vdots \\ \hat{\mathbf{x}}_{k+H} \end{bmatrix} \in \mathbb{R}^{H \times D_{\text{out}}}. \quad (9)$$

1.5 Prediction Outputs

The neural model predicts state increments

$$\Delta \hat{\mathbf{Y}}_{k+1:k+H}^{(H)} \in \mathbb{R}^{H \times D_{\text{out}}}, \quad (10)$$

and, when the uncertainty branch is enabled, diagonal log-variance terms

$$\log \hat{\boldsymbol{\sigma}}_{k+1:k+H}^2 \in \mathbb{R}^{H \times D_{\text{out}}}. \quad (11)$$

For the current multi-step branch, rollout uses the last proxy state in the history window as initial condition:

$$\hat{\mathbf{x}}_{k+j} = \hat{\mathbf{x}}_k + \sum_{i=1}^j \Delta \hat{\mathbf{x}}_{k+i}, \quad j = 1, \dots, H. \quad (12)$$

For the single-step branch, $H = 1$, so the predictor is interpreted as a direct approximation of a one-step transition operator.

2 Data Preprocessing and Proxy-State Construction

2.1 Why Preprocessing Must Be Part of the Theory

For the present project, data preprocessing is not a secondary engineering detail. It determines whether the downstream learning problem can be interpreted as a causal state-transition identification problem.

If one trains directly on raw asynchronous sensor streams, the network must jointly solve:

- multi-rate temporal alignment,
- denoising and bias suppression,
- sparse-observation completion,
- and controlled dynamics learning.

This coupling makes the optimization target ill-posed for short-horizon rollout. Therefore, the preprocessing pipeline must be described explicitly in the mathematical method section.

2.2 Raw Sensor Streams

The raw data sources are:

- PWM commands at 100 Hz,
- IMU measurements at 100 Hz,
- DVL velocity observations at approximately 10 Hz,
- motor power observations at approximately 5 Hz.

These signals do not arrive on a common time axis and do not play the same role in the transition problem. Representative raw measurements are shown in Fig. 1.

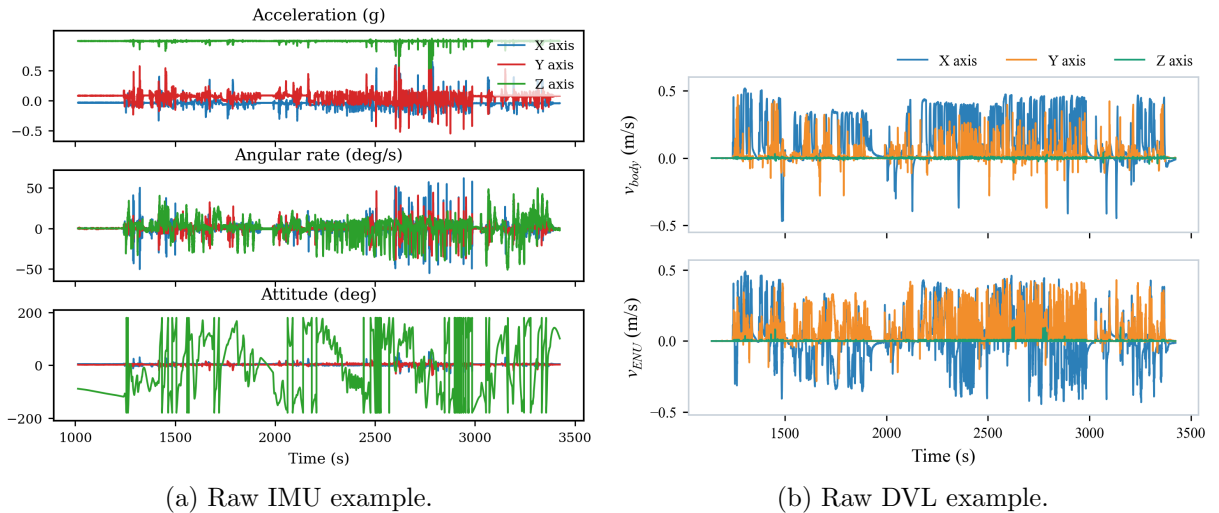


Figure 1: Representative raw sensor observations from the pool test dataset. The IMU provides dense body-motion evidence, while the DVL supplies lower-rate velocity observations.

2.3 IMU Preprocessing

Let the accelerometer and gyroscope observations be denoted by

$$\tilde{\mathbf{a}}_k = \mathbf{a}_k - \mathbf{R}_{bn,k}\mathbf{g} + \mathbf{b}_{a,k} + \mathbf{n}_{a,k}, \quad (13)$$

$$\tilde{\boldsymbol{\omega}}_k = \boldsymbol{\omega}_k + \mathbf{b}_{\omega,k} + \mathbf{n}_{\omega,k}, \quad (14)$$

where $\mathbf{R}_{bn,k}$ is the body-to-navigation rotation, $\mathbf{g}$ is gravity, $\mathbf{b}_{a,k}$ and $\mathbf{b}_{\omega,k}$ are sensor biases, and $\mathbf{n}_{a,k}, \mathbf{n}_{\omega,k}$ denote measurement noise.

The IMU preprocessing chain applies:

1. frame transformation into the training body frame,
2. gravity compensation,
3. bias correction,
4. and light filtering for numerical stability.

The purpose is not to create a perfect inertial solution, but to provide a cleaner high-rate propagation signal for the subsequent causal fusion stage. An example of the processed IMU output is shown in Fig. 2.

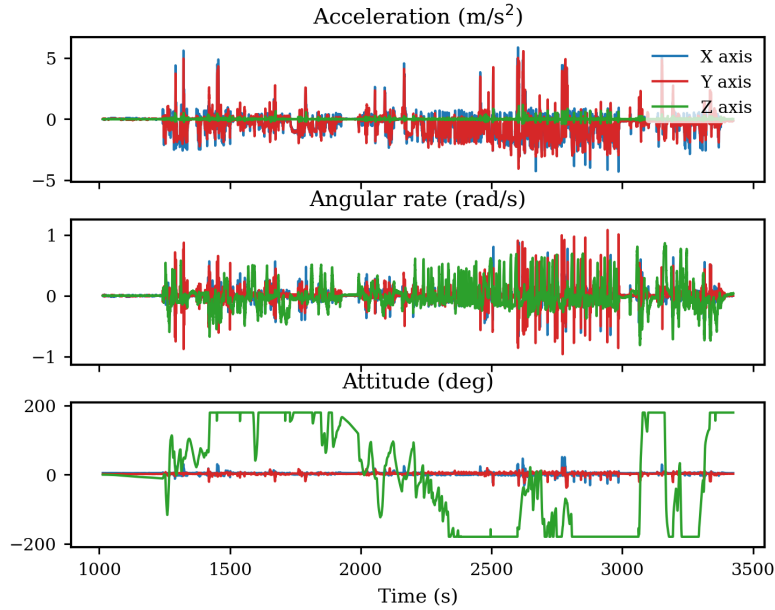


Figure 2: Example of the IMU preprocessing output after frame conversion, gravity handling, bias correction, and filtering.

2.4 Multi-Rate Alignment

All observations are projected onto a common master axis with sampling interval $\Delta t = 0.01$ s. This aligned axis serves two purposes:

- it makes the control input and state proxy share the same temporal semantics;
- it converts the learning problem into a discrete-time transition problem indexed by k .

The aligned training table therefore acts as the bridge between raw logging and transition-model learning.

2.5 Causal KF / ESKF Fusion

After alignment, a causal KF / ESKF stage combines high-rate IMU propagation with sparse DVL corrections. The propagation step can be written abstractly as

$$\hat{\mathbf{s}}_{k|k-1} = f_{\text{imu}}(\hat{\mathbf{s}}_{k-1|k-1}, \tilde{\mathbf{a}}_k, \tilde{\boldsymbol{\omega}}_k), \quad (15)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (16)$$

where $\hat{\mathbf{s}}_{k|k-1}$ denotes the predicted internal state and $\mathbf{P}_{k|k-1}$ its covariance.

When a valid DVL measurement is available, the velocity-related update is applied:

$$\mathbf{z}_k^v = \mathbf{H}_k \hat{\mathbf{s}}_{k|k-1} + \mathbf{r}_k, \quad (17)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \left(\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}_k \right)^{-1}, \quad (18)$$

$$\hat{\mathbf{s}}_{k|k} = \hat{\mathbf{s}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k^v - \mathbf{H}_k \hat{\mathbf{s}}_{k|k-1}). \quad (19)$$

The output of this stage is not a full navigation solution for external deployment. Instead, it is a causally estimated training-oriented proxy state:

$$\hat{\mathbf{x}}_k = \begin{bmatrix} \hat{\mathbf{a}}_k \\ \hat{\boldsymbol{\omega}}_k \\ \hat{\mathbf{v}}_k \end{bmatrix}, \quad (20)$$

together with attitude context and reliability-related quantities such as DVL freshness and velocity covariance proxies.

2.6 From Raw Observations to Dense Proxy States

The effect of the fusion stage is illustrated in Fig. 3. The dense IMU stream supplies propagation information, while the sparse DVL observations intermittently correct the velocity estimate. The result is a dense proxy-state sequence that remains compatible with online rollout semantics.

2.7 Full-Run Visual Audit

Short windows are useful for local interpretation, but they are insufficient for judging whether preprocessing quality is stable over an entire experiment. For this reason, the current documentation also keeps a full-run visual audit, shown in Fig. 4.

This full-run figure serves three purposes:

- it checks whether the KF proxy remains bounded over the whole trajectory,
- it reveals whether DVL corrections are frequent enough to suppress inertial drift,
- it helps identify non-physical spikes, rejected measurements, and long segments with stale velocity correction.

2.8 Training Precursor Interpretation

The preprocessing pipeline leads to the following modeling stance:

- PWM is treated as the primary control input,
- $\hat{\mathbf{x}}_k$ is treated as the causal proxy state,
- attitude and quality fields are treated as context variables.

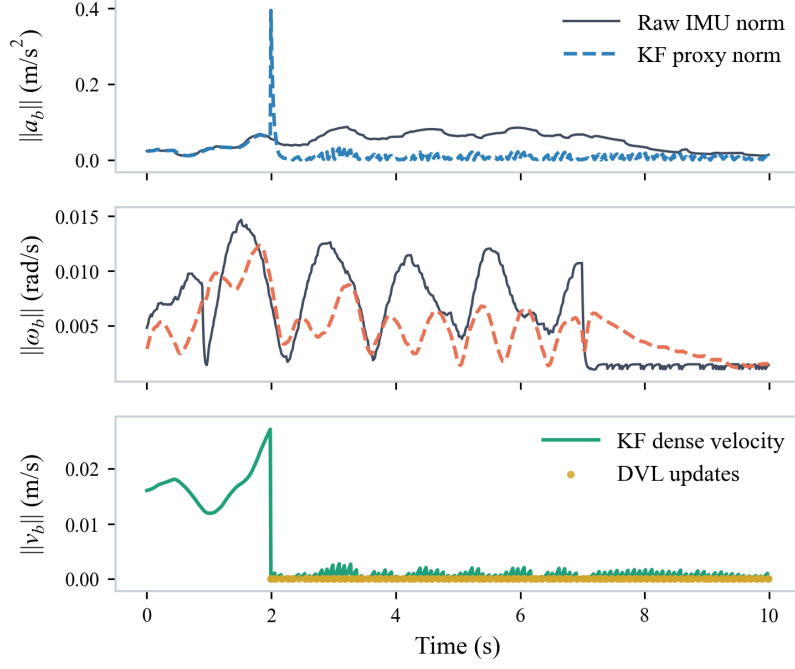


Figure 3: Example of proxy-state construction on the aligned training base table. The KF proxy suppresses raw IMU noise and turns sparse DVL velocity updates into a dense rollout-compatible velocity state.

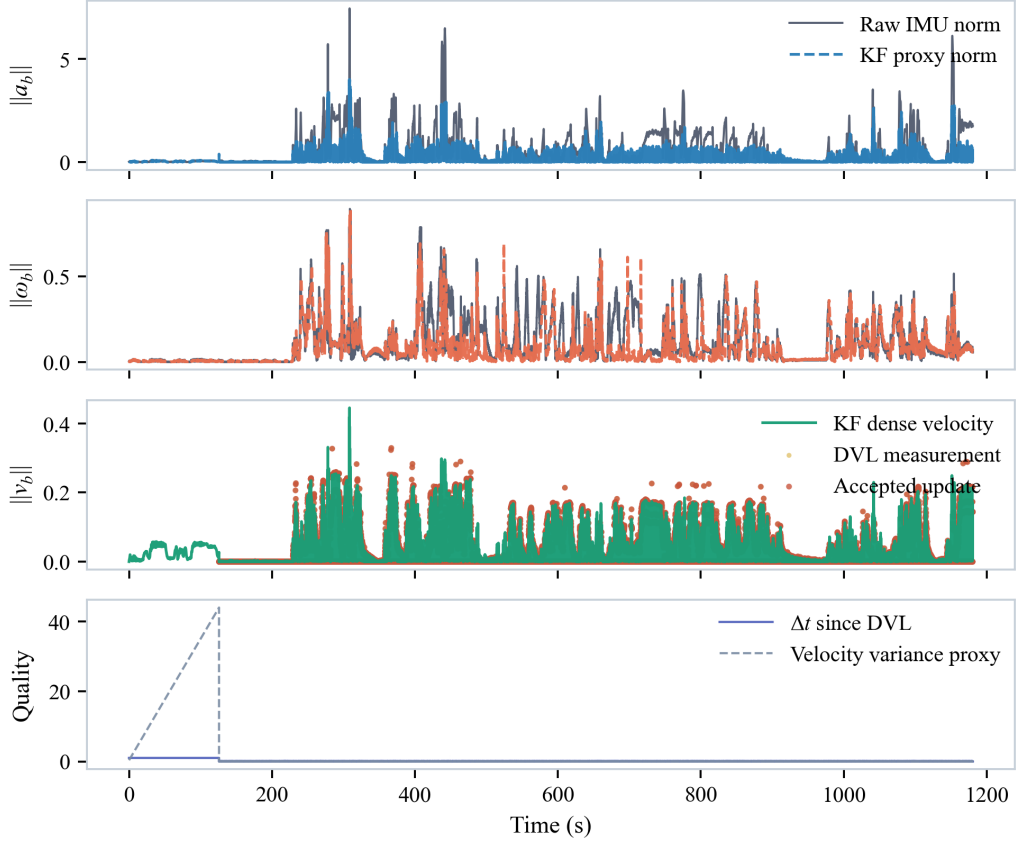


Figure 4: Full-run visual audit of the preprocessing and fusion output. This figure is intended for method validation rather than presentation-only illustration, because it exposes drift, correction freshness, variance growth, and possible outlier behavior over the complete experiment.

Hence, the network is not trained to recover state directly from raw heterogeneous observations. Instead, it is trained on

$$\mathbf{z}_k = \begin{bmatrix} \mathbf{u}_k \\ \hat{\mathbf{x}}_k \\ \mathbf{c}_k \end{bmatrix}, \quad (21)$$

which is mathematically much closer to a discrete-time controlled state-transition problem.

This preprocessing-aware viewpoint is the reason the current project can move from sequence fitting toward a solver-style model:

$$\hat{\mathbf{x}}_{k+1} = f_\theta(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k). \quad (22)$$

3 Underwater Dynamics Model and Motivation

3.1 Classical Dynamics Background

A standard body-frame underwater-vehicle dynamics model may be written as

$$\mathbf{M}(\boldsymbol{\nu})\dot{\boldsymbol{\nu}} + \mathbf{C}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{D}(\boldsymbol{\nu})\boldsymbol{\nu} + \mathbf{g}(\boldsymbol{\eta}) = \boldsymbol{\tau}, \quad (23)$$

where $\boldsymbol{\nu}$ denotes generalized body-frame velocity, $\boldsymbol{\eta}$ denotes pose, and $\boldsymbol{\tau}$ denotes generalized actuation.

This form is physically meaningful, but it is difficult to identify accurately in the present engineering setting because:

- the thruster response is nonlinear and affected by loading history,
- hydrodynamic effects vary with operating condition and maneuver regime,
- the available observations are asynchronous and partially missing,
- the immediate engineering target is short-horizon prediction for control rather than full physical coefficient recovery.

3.2 Control-Oriented Identification Objective

The present project therefore adopts a control-oriented data-driven surrogate. Instead of recovering all physical coefficients, we seek a transition map of the form

$$\hat{\mathbf{x}}_{k+1} = f_\vartheta(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k), \quad (24)$$

or, equivalently, an increment form

$$\Delta\hat{\mathbf{x}}_k = g_\vartheta(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k), \quad \hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_k + \Delta\hat{\mathbf{x}}_k. \quad (25)$$

When trained in block-prediction mode, the same idea is expressed as

$$\hat{\mathbf{X}}_{k+1:k+H} \approx f_\vartheta(\mathbf{X}_k^{(L)}), \quad (26)$$

but the intended physical interpretation remains that of a short-horizon controlled transition model.

3.3 Why Proxy-State Learning Is More Representative Than Raw-Sensor Fitting

Let the raw observation stream be denoted by $\mathcal{Y}_{0:k}$. If a neural network is trained directly from raw sensor histories, it must simultaneously learn:

- temporal alignment across different rates,
- denoising and bias suppression,
- interpolation across sparse DVL updates,
- and the actual controlled dynamics.

This makes the optimization target poorly conditioned. The network is no longer solving only a dynamics-identification problem; it is also solving a hidden state-estimation problem.

By contrast, the current project introduces a causal estimator

$$\hat{\mathbf{x}}_k = \mathcal{F}(\mathcal{Y}_{0:k}, \mathbf{u}_{0:k}), \quad (27)$$

and then learns the transition law on this proxy-state manifold. This changes the learning problem from “raw observation fitting” to “state-proxy transition fitting,” which is more representative for control and simulation because:

- the state proxy is already aligned to the control clock,
- the target lives in a body-frame state space that can be recursively rolled out,
- low-rate DVL information is absorbed as correction rather than treated as irregular labels,
- the resulting model is closer to the interface required by controller and RL algorithms.

3.4 Role of Causal Filtering

The KF / ESKF stage is therefore not merely a denoising preprocessor. It is a causal state-construction layer that makes the downstream identification problem better posed.

The roles of the sensors are distinct:

- IMU provides dense propagation information,
- DVL provides sparse but strong velocity correction,
- Power provides auxiliary evidence about effective actuation.

Consequently, the learned state should not be equated with any single sensor. It should be interpreted as a causally estimated state proxy that preserves the dominant short-horizon transition structure relevant to control.

3.5 Weak Structural Priors

The current architecture also allows weak structured priors between pure black-box fitting and full white-box identification:

- actuator-side lag and saturation effects,
- latent hydrodynamic memory,
- damping-related output bias,
- heteroscedastic predictive uncertainty.

These priors are not meant to recover a complete physical model. Their role is to regularize the learned transition operator so that it better reflects controllable short-horizon dynamics.

4 Neural Dynamics Model

4.1 History Encoder

Let the history window be

$$\mathbf{X}_k^{(L)} \in \mathbb{R}^{L \times D_{\text{in}}}. \quad (28)$$

The current baseline model family is **S1Predictor**, which uses a sequence encoder to summarize the control and proxy-state history:

$$\mathbf{h}_k = \Phi_\theta(\mathbf{X}_k^{(L)}). \quad (29)$$

In the current mainline implementation, Φ_θ is typically instantiated as an LSTM encoder.

4.2 Grouped Prediction Head

Conditioned on $\mathbf{h}_k$, the prediction head produces future state increments:

$$\Delta \hat{\mathbf{Y}}_{k+1:k+H} = \Psi_\theta(\mathbf{h}_k) \in \mathbb{R}^{H \times D_{\text{out}}}. \quad (30)$$

When enabled, the uncertainty head additionally predicts

$$\log \hat{\sigma}_{k+1:k+H}^2 = \Omega_\theta(\mathbf{h}_k) \in \mathbb{R}^{H \times D_{\text{out}}}. \quad (31)$$

The grouped-head design reflects the current engineering prior that acceleration, angular rate, and velocity should remain distinguishable state groups during optimization and evaluation.

4.3 Multi-Step Block Predictor

For the current multi-step mainline, the model predicts a block of future increments with horizon $H > 1$:

$$\left(\Delta \hat{\mathbf{Y}}_{k+1:k+H}, \log \hat{\sigma}_{k+1:k+H}^2 \right) = f_\theta(\mathbf{X}_k^{(L)}). \quad (32)$$

Let the last proxy state inside the history window be denoted by $\hat{\mathbf{x}}_k$. The predicted future state sequence is reconstructed by delta-cumsum rollout:

$$\hat{\mathbf{x}}_{k+j} = \hat{\mathbf{x}}_k + \sum_{i=1}^j \Delta \hat{\mathbf{x}}_{k+i}, \quad j = 1, \dots, H. \quad (33)$$

This contract is used by the current training and evaluation pipelines because it preserves direct comparability with existing long-rollout metrics and artifacts.

4.4 Single-Step Transition Branch

To move closer to an explicit state-transition formulation, the project also maintains a single-step branch with $H = 1$:

$$\Delta \hat{\mathbf{x}}_{k+1} = f_\theta^{\text{step}}(\mathbf{X}_k^{(L)}), \quad \hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_k + \Delta \hat{\mathbf{x}}_{k+1}. \quad (34)$$

Although the implementation still consumes a history window, the predictive semantics are much closer to

$$\hat{\mathbf{x}}_{k+1} = f_\vartheta(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k), \quad (35)$$

because the output is restricted to a one-step update.

4.5 Optional Structured Modules

The network may include additional structured modules:

- **ThrusterLag**: transforms raw command history into effective control features,
- **HydroSSM**: provides latent hydrodynamic memory,
- **DampingHead**: injects damping-related inductive bias into predicted increments,
- **UncertaintyHead**: predicts diagonal log-variance terms.

Accordingly, one may interpret the overall model abstractly as

$$\left(\Delta\hat{\mathbf{Y}}, \log \hat{\boldsymbol{\sigma}}^2\right) = f_{\theta}\left(\mathbf{X}_k^{(L)}, \mathbf{r}_k\right), \quad (36)$$

where $\mathbf{r}_k$ denotes optional structured latent features induced by these modules.

4.6 Architectural Interpretation

The current architecture should therefore be understood as a transition-model family with two active semantic modes:

- a multi-step block predictor used to preserve evaluation continuity,
- a single-step transition branch used to approach solver-style deployment.

This distinction is important for both engineering and scientific writing: the former is the current operational baseline, while the latter is the mathematically cleaner path toward controller and RL integration.

5 Training Objective

5.1 Normalized Training Space

Let the proxy state be normalized component-wise as

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}. \quad (37)$$

Because the rollout is increment-based, every state component shared between the history input and the target must use the same normalization statistics. Otherwise, the recurrence

$$\tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{x}}_k + \Delta\tilde{\mathbf{x}}_{k+1} \quad (38)$$

would lose its physical interpretation in the original state space.

5.2 Gaussian Output Objective

When the uncertainty branch is enabled, the predictor outputs both mean increments and diagonal log-variance terms. After rollout reconstruction, the baseline probabilistic loss is a diagonal Gaussian negative log-likelihood:

$$\mathcal{L}_{\text{nll}} = \sum_{j=1}^H \sum_{d=1}^{D_{\text{out}}} \left[\frac{(\hat{x}_{k+j,d} - x_{k+j,d})^2}{\hat{\sigma}_{k+j,d}^2} + \log \hat{\sigma}_{k+j,d}^2 \right]. \quad (39)$$

5.3 Mask-Aware Supervision

Even after proxy-state construction, some observation-derived quantities remain reliability-dependent. This motivates a generic mask-aware form:

$$\mathcal{L}_{\text{mask}} = \sum_{j=1}^H \sum_{d=1}^{D_{\text{out}}} m_{k+j,d} \left[\frac{(\hat{x}_{k+j,d} - x_{k+j,d})^2}{\hat{\sigma}_{k+j,d}^2} + \log \hat{\sigma}_{k+j,d}^2 \right], \quad (40)$$

where $m_{k+j,d} \in \{0, 1\}$ denotes whether the supervision at that step and dimension is valid.

In the current KF-proxy mainline, velocity-state supervision has already been densified through causal fusion, so the engineering importance of masking has shifted from “sparse DVL labels” to “reliability-aware metric interpretation and future observation-consistency extensions”.

5.4 Transition-Balance Objective

The current mainline optimization focus is not plain NLL minimization. Instead, the project uses a transition-oriented objective that keeps a generic state-regression term while emphasizing state-group balance, horizon-tail behavior, and increment consistency.

In abstract form, the loss can be written as

$$\mathcal{L}_{\text{tb}} = \lambda_{\text{mse}} \mathcal{L}_{\text{mse}} + \lambda_{\text{state}} \mathcal{L}_{\text{state}} + \lambda_{\Delta} \mathcal{L}_{\Delta} + \lambda_{\text{tail}} \mathcal{L}_{\text{tail}}, \quad (41)$$

where:

- $\mathcal{L}_{\text{mse}}$ is a generic rollout-state mean-squared error term,
- $\mathcal{L}_{\text{state}}$ penalizes rolled-out state error,
- $\mathcal{L}_{\Delta}$ penalizes increment mismatch,
- $\mathcal{L}_{\text{tail}}$ upweights later horizon steps to suppress long-rollout drift.

In practice, the state components are also grouped as

$$\hat{\mathbf{x}}_k = [\hat{\mathbf{a}}_k \quad \hat{\boldsymbol{\omega}}_k \quad \hat{\mathbf{v}}_k]^\top, \quad (42)$$

so that acceleration, angular-rate, and velocity errors can be weighted separately.

5.5 Single-Step Interpretation

For the single-step branch, $H = 1$, so the same transition-oriented loss simplifies to a one-step objective:

$$\mathcal{L}_{\text{step}} = \lambda_{\text{state}} \|\hat{\mathbf{x}}_{k+1} - \mathbf{x}_{k+1}\|^2 + \lambda_{\Delta} \|\Delta \hat{\mathbf{x}}_{k+1} - \Delta \mathbf{x}_{k+1}\|^2. \quad (43)$$

This is mathematically closer to system identification in state-space form and is therefore the more natural candidate for later solver-style deployment.

5.6 Model Selection Criterion

The training objective and the model-selection objective are related but not identical. The current repository additionally tracks a validation metric oriented toward transition quality, denoted conceptually by

$$\text{Score}_{\text{trans}} = \alpha_1 E_{\text{global}} + \alpha_2 E_{\text{final}} + \alpha_3 E_{\text{tail}} + \alpha_4 E_{\text{growth}}, \quad (44)$$

where the individual terms summarize global rollout error, final-step error, tail-horizon error, and rollout growth.

In addition, the training loop now records generic validation-space RMSE and MAE in normalized space, so that optimization can be audited not only through transition-specific terms but also through conventional regression error.

This distinction matters because a model that attains low average loss may still be unsuitable as a transition solver if it exhibits unstable tail behavior.

6 Feasibility of a Learned Transition Solver

6.1 Problem Statement

The current project goal is not to replace a full high-fidelity hydrodynamic model. Instead, the learned model is expected to serve as a short-horizon state transition solver inside a controller, a lightweight simulator, or a reinforcement-learning environment.

Let the latent engineering state be

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{a}_k \\ \boldsymbol{\omega}_k \\ \mathbf{v}_k \end{bmatrix} \in \mathbb{R}^9,$$

where $\mathbf{a}_k$ is body-frame linear acceleration, $\boldsymbol{\omega}_k$ is body-frame angular rate, and $\mathbf{v}_k$ is body-frame velocity.

The control-related input is

$$\mathbf{u}_k \in \mathbb{R}^8,$$

and the auxiliary context is

$$\mathbf{c}_k = [\phi_k \quad \theta_k \quad \sin \psi_k \quad \cos \psi_k \quad \mathbf{p}_k]^\top,$$

where ϕ_k, θ_k, ψ_k are attitude terms and $\mathbf{p}_k$ denotes motor power observations.

The desired one-step transition operator is

$$\mathbf{x}_{k+1} = f_\vartheta(\mathbf{x}_k, \mathbf{u}_k, \mathbf{c}_k),$$

or equivalently in increment form

$$\Delta \mathbf{x}_k = g_\vartheta(\mathbf{x}_k, \mathbf{u}_k, \mathbf{c}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k.$$

6.2 Why Sensor Preprocessing is Necessary

Raw asynchronous sensor streams do not directly satisfy the assumptions required by a transition model:

1. The observations arrive at different rates.
2. DVL velocity is sparse and intermittent.
3. IMU signals are dense but noisy and bias-sensitive.
4. Power observations are low-rate auxiliary measurements rather than primary state labels.

If these signals are fed directly into a neural predictor, the network must simultaneously solve:

1. time alignment,

2. denoising and bias suppression,
3. state estimation,
4. and true controlled dynamics learning.

This overloads the model and usually leads to unstable rollout.

6.3 Causal Filtering Makes the Learning Problem Better Posed

Suppose a causal estimator produces a proxy state

$$\hat{\mathbf{x}}_k = \mathcal{F}(\mathcal{Y}_{0:k}, \mathbf{u}_{0:k}),$$

where $\mathcal{Y}_{0:k}$ contains all sensor observations available up to time k .

The key requirement is causality:

$$\hat{\mathbf{x}}_k = \mathcal{F}(\mathcal{Y}_{0:k}, \mathbf{u}_{0:k}),$$

that is, the estimate at time k must not depend on future observations $\mathcal{Y}_{k+1:T}$. This requirement is essential because the downstream transition model is intended for online rollout.

Under this setup, the learning target becomes

$$\hat{\mathbf{x}}_{k+1} \approx f_{\vartheta}(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k),$$

which is a standard output-error system identification problem with filtered state proxies.

This is mathematically feasible as long as the filtered proxy states satisfy two properties:

1. They are sufficiently low-noise to reduce label variance.
2. They preserve the dominant short-horizon transition structure relevant to control.

The existing KF / ESKF preprocessing chain is therefore not a cosmetic improvement. It reduces the estimation burden seen by the neural transition model.

6.4 Why Proxy-State Training Is More Representative

For a transition solver, the decisive property is not whether the training labels preserve every sensor-side detail, but whether they preserve the controlled short-horizon evolution relevant to recursive prediction.

The proxy-state formulation is more representative than raw-sensor fitting because:

1. it places both input and output on the same aligned time base;
2. it expresses the learned quantity in a body-frame state space compatible with recursive rollout;
3. it separates “state construction” from “transition learning”;
4. it allows reliability and freshness information to enter the model explicitly through context variables.

Hence, the learned predictor is no longer asked to imitate a heterogeneous observation stream. It is asked to approximate a state transition law on a causally estimated state manifold.

6.5 Role of DVL, IMU, and Power

The sensors have different mathematical roles:

- IMU provides dense propagation information.
- DVL provides sparse but strong velocity correction.
- Power provides auxiliary evidence about actuator loading and effective thrust.

Hence, the engineering state should not be equated with any single sensor. Instead, it should be regarded as a causally estimated proxy state.

This also explains why power is better treated as context or auxiliary observation rather than as a direct state target: power helps constrain the map from command to effective actuation, but it is not itself the rigid-body motion state.

6.6 Why Shared Scaling is a Hard Requirement

Assume the rollout is trained in normalized space:

$$\tilde{\mathbf{x}} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}.$$

If the initial state $\mathbf{x}_k$ is normalized by one scaler and the target $\mathbf{x}_{k+1}$ is normalized by another scaler, then the recurrence

$$\tilde{\mathbf{x}}_{k+1} = \tilde{\mathbf{x}}_k + \Delta\tilde{\mathbf{x}}_k$$

no longer corresponds to a consistent physical-space update.

Therefore, for every shared state component used both in the history input and in the prediction target, the same normalization statistics must be used.

This is not a minor implementation detail. It is a necessary condition for the increment-based rollout semantics to remain valid.

6.7 Phase 1 Feasibility Claim

After enforcing:

1. causal filtered state proxies, and
2. shared normalization semantics for overlapping state dimensions,

the project has a defensible foundation for learning a short-horizon transition solver.

What Phase 1 proves is:

- the training target can be interpreted as a causal state-proxy transition problem;
- the rollout objective is numerically coherent;
- the model can be upgraded toward a one-step solver without rebuilding the whole data chain.

What Phase 1 does *not* prove is:

- global closed-loop stability,
- optimal controller compatibility,
- or equivalence to a high-fidelity physical simulator.

6.8 Current Engineering Interpretation

The present repository still contains both:

- a multi-step block-prediction line for horizon-comparable offline evaluation, and
- a single-step transition line that is closer to a solver-style recursive interface.

The current engineering preference is the second line. This preference does not come from aesthetics; it comes from the project requirement that the model be usable as a short-horizon transition component inside replay, control-readiness analysis, and later controller / RL integration.

Therefore, the most relevant mathematical question is no longer “how far can a fixed future block be predicted,” but rather “whether a causally constructed proxy state admits a stable short recursive transition law.”

6.9 Next Mathematical Step

The next step is to consolidate the existing single-step transition branch and continue shifting from block prediction toward explicit one-step transition learning:

$$\hat{\mathbf{x}}_{k+1} = f_{\vartheta}(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k),$$

with short recursive unrolling during training.

The corresponding loss should combine:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{step}} + \lambda_2 \mathcal{L}_{\text{rollout}} + \lambda_3 \mathcal{L}_{\text{obs}} + \lambda_4 \mathcal{L}_{\text{stability}},$$

where:

- $\mathcal{L}_{\text{step}}$ enforces one-step transition accuracy,
- $\mathcal{L}_{\text{rollout}}$ enforces short recursive consistency,
- $\mathcal{L}_{\text{obs}}$ enforces agreement with sensor-space observations,
- $\mathcal{L}_{\text{stability}}$ penalizes divergence, non-finite behavior, or excessive tail growth.

This is the mathematically natural bridge from offline fitted sequence prediction to a transition model usable by replay screening, control-oriented validation, and reinforcement learning.

7 Control Integration

7.1 Target Role of the Learned Model

The long-term target is to use the learned predictor as a short-horizon transition model inside a controller, a lightweight simulator, or a model-based reinforcement-learning loop. The intended interface is therefore not merely “predict the next logged trajectory,” but

$$\hat{\mathbf{x}}_{k+1:k+H} = f_{\theta}(\hat{\mathbf{x}}_k, \mathbf{u}_{k:k+H-1}, \mathbf{c}_k), \quad (45)$$

possibly together with uncertainty and reliability information.

7.2 Why Offline Screening Is Still Necessary

At the current project stage, the repository should not claim verified closed-loop readiness. Before the predictor is inserted into a control loop, it must first satisfy offline criteria related to numerical stability and transition fidelity.

The current offline metrics therefore act as a screening layer:

- global error terms such as RMSE and MAE,
- final-step error,
- rollout-growth measures,
- tail-horizon error statistics,
- worst-component bias and non-finite-event counts.

These metrics do not prove closed-loop stability, but they do provide evidence about whether the learned model can serve as a plausible short-horizon solver candidate.

7.3 Controller-Oriented Objective

If the predictor is later used inside a finite-horizon controller, a generic objective can be written as

$$\min_{\mathbf{u}_{k:k+H-1}} \sum_{j=0}^{H-1} \left(\|\hat{\mathbf{x}}_{k+j} - \mathbf{x}_{k+j}^{\star}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_{k+j}\|_{\mathbf{R}}^2 \right), \quad (46)$$

subject to actuator and safety constraints.

In this context, the learned model is valuable only if its short-horizon rollout remains numerically stable and sensitive to the applied control sequence.

7.4 Why the Single-Step Branch Matters

The single-step transition branch is strategically important for control integration because it more directly matches the recursive computation required by control and simulation:

$$\hat{\mathbf{x}}_{k+1} = f_{\theta}^{\text{step}}(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k). \quad (47)$$

Compared with a fixed future-block predictor, this form offers:

- cleaner solver semantics,
- easier online recursion,
- clearer controller and RL interface design,
- more direct latency accounting in closed-loop evaluation.

7.5 Current Boundary of Claims

The present repository does *not* yet establish:

- closed-loop stability guarantees,
- high-fidelity physical-simulator equivalence,
- or deployment-grade safety certification.

What it does establish is a mathematically and engineeringly coherent bridge from offline sequence fitting to transition-solver evaluation:

- causal proxy-state construction,
- transition-oriented training and selection,
- artifact-based offline screening,
- and a clear path toward controller-compatible interfaces.

8 Interface to Control Algorithms

8.1 Minimal Solver API

For controller, simulator, and model-based RL integration, the learned model should eventually expose a minimal solver-style interface:

$$\text{reset}(\hat{\mathbf{x}}_0, \mathbf{c}_0) \rightarrow \text{internal state}, \quad (48)$$

$$\text{step}(\mathbf{u}_k, \Delta t, \mathbf{c}_k) \rightarrow (\hat{\mathbf{x}}_{k+1}, \mathbf{i}_{k+1}), \quad (49)$$

$$\text{rollout}(\mathbf{u}_{k:k+H-1}, \mathbf{c}_{k:k+H-1}) \rightarrow \hat{\mathbf{x}}_{k+1:k+H}. \quad (50)$$

Here, $\mathbf{i}_{k+1}$ denotes diagnostic information returned together with the predicted next state.

8.2 Diagnostic Information

The diagnostic payload should include at least:

- sensor freshness indicators,
- quality-context values such as elapsed time since the last DVL correction,
- predictive uncertainty or confidence surrogates when available,
- non-finite-event flags and fallback triggers,
- timing information for control-loop accounting.

This is important because a solver that predicts a state value without exposing reliability information is difficult to use safely inside controller or RL loops.

8.3 Compatibility With Controller and RL Loops

In a controller loop, the model is recursively queried as

$$\hat{\mathbf{x}}_{k+1} = f_{\theta}(\hat{\mathbf{x}}_k, \mathbf{u}_k, \mathbf{c}_k), \quad (51)$$

and the resulting state is used for cost evaluation, policy updates, or search over future actions.

In model-based RL, the same interface supports:

- imagined rollouts,
- short-horizon planning,
- synthetic trajectory generation for policy improvement.

Therefore, the main engineering requirement is not a visually impressive trajectory plot, but a numerically stable and well-instrumented transition step.

8.4 Current Repository Status

The repository does not yet expose a fully unified production-ready `step(u_t, dt)` interface. However, the present data contract, the single-step transition branch, and the offline transition-solver wrapper already define the mathematical form such an interface should follow.

In other words, the project is no longer only discussing a solver in the abstract: it already has the ingredients for offline replay-style solver validation, but it has not yet completed the final interface unification needed for controller deployment.

This makes the current phase a preparation stage for solver-style deployment rather than a terminal modeling stage.